

Python – Resumen

Bruno Motto

Python es un lenguaje de alto nivel de programación interpretado cuya filosofía hace hincapié en la legibilidad de su código. Se trata de un lenguaje de programación multiparadigma (de propósito general), ya que soporta parcialmente la orientación a objetos, programación imperativa y, en menor medida, programación funcional. Además tiene tipado dinámico, es decir que las variables se amoldan a los datos. El código de Python es interpretado lee instrucción por instrucción o línea por línea. (De arriba hacia abajo). Este lenguaje trabaja con indentación, lo que quiere decir que se debe seguir cierta estructura para que las cosas tengan sentido (es decir ser ordenado con cada espacio que dejamos).

INGRESAR DATOS:

En Python, la entrada de datos se realiza a través de la función `input()`. Esta función permite al usuario ingresar datos desde el teclado. Aquí, la función `input()` muestra un mensaje en la consola, pidiéndole al usuario que ingrese un valor.

```
radio = float(input("Introduce el radio de la base del cilindro: "))
altura = float(input("Introduce la altura del cilindro: "))
```

IMPRIMIR DATOS:

Para que python nos muestre texto o variables hay que utilizar la función `print()`. Las cadenas se pueden delimitar tanto por comillas dobles (") como por comillas simples (').

```
print("horas: ",horas," minutos: ",minutos," segundos: ", segundos)
```

A su vez, si queremos colocar una variable dentro de las comillas podemos colocar una `f` antes de la primer comilla y luego incluir la variable entre llaves.

```
print(f"El volumen del cilindro es: {volumen}")
```

WHILE: En Python para repetir una secuencia con `while` debemos colocar la condición entre paréntesis, luego colocar 2 puntos y a partir de ahí todo lo que ingresemos desplazado a la derecha con tab será la secuencia a repetir hasta que la condición deje de cumplirse.

```
i=1
while(i<=N):
    print(i)
    i+=1
```

FOR: Cuando utilizamos el `for` en Python indexamos una variable a una estructura, algo similar al conocido `for x`, en el caso de que la estructura sea una lista, vector o cualquiera, la sintaxis será "for variable in Nombre_estructura:" y luego el bucle, un caso particular muy usado es cuando queremos que esta variable vaya creciendo en un rango para esto la estructura será `range(desde,hasta,con paso)`. Si no colocamos desde el inicio será en 0, si no colocamos el paso este será de 1, es importante saber que no incluye el hasta (es decir funciona mientras sea menor).

```
for i in l:
    print(i)
```

```
for i in range(1,N+1,2):
    print(i)
```

IF: Se coloca la condición, luego 2 puntos “:”, abajo desplazado a la derecha colocamos las acciones en caso de que la condición sea verdadera. La diferencia con otros lenguajes es que luego para colocar lo que sucede si no se cumple la condición tenemos 2 opciones, si además queremos que se cumpla otra condición para que se ejecute el código en caso de falso en la primer condicion, usaremos ELIF, seguido de la nueva condición y “:”, si no debemos agregar ninguna condición podemos simplemente usar ELSE.

```
if(IMC<18.5):  
    print("Tienes bajo peso")  
elif(IMC<24.9):  
    print("Tienes un peso saludable")  
elif(IMC<29.9):  
    print("Tienes sobrepeso")  
elif(IMC<39.9):  
    print("Tienes obesidad")  
else: print ("Tienes obesidad severa")
```

AGREGAR LIBRERÍAS: Para agregar librerías debemos escribir antes del código import nombre_libreria

```
import math
```

BUSCAR Y REMPLAZAR (VSC): Control+f para buscar y Control+h para remplazar. (Control+Alt+Enter para remplazar todas las apariciones).

BIBLIOTECA ESTANDAR DE PYTHON: <https://docs.python.org/es/3/library/index.html>

DESARROLLO DE SITIOS WEB CON PYTHON: Se utilizan dos grandes frameworks, Flask y django.

HACER CORRER LA CONSOLA: Tienes que estar en la sección RUN AND DEBUG y dar click al F5, si escribis clear se limpia la consola.

LISTAS: Las listas en Python son dinámicas y pueden contener cualquier tipo de datos, lo que las hace muy versátiles. Estas pueden contener una colección ordenada de elementos, incluso de elementos de diferentes tipos de datos(listas mixtas). Se pueden crear listas utilizando corchetes [] y separando los elementos por comas.

```
lista_vacia=[]  
lista_mixta=["Carlos",17,"Juan",20]
```

Se les puede **agregar elementos al final** utilizando la función NombreLista.append(dato)

```
lista_mixta.append("Miguel")
```

Se puede acceder a los elementos de la lista por su índice, ya sea para leerlos o modificarlos.

Para acceder desde el principio arrancamos con 0, si queremos arrancar por el final podemos usar números negativos desde -1.

Para saber el **tamaño de una lista** utilizaremos la función len(NombreLista)

DEFINIR MAIN:

```
if (__name__ == "__main__"):
```

POO EN PYTHON:

CLASES: Para crear una clase debemos escribir `class NombreClase():`

A menos que creamos un constructor en cuyo caso no pondremos los paréntesis ().

Dentro definimos todos los atributos y métodos que tendrá la clase. Para agregar un atributo simplemente escribimos NombreAtributo=ValorAtributo (Cuando no usamos constructor propio).

CONSTRUCTOR: Para crear un constructor debemos crear un método de la siguiente forma:
def __init__(self, parametro1, parametro2, etc):

Self es una forma de hacer referencia a si mismo, es decir al propio objeto.

Luego dentro del método constructor debemos crear los atributos, de la forma
self.atributo1=parametro1

Para crear un objeto debemos instanciar la clase utilizando su constructor (ya sea el constructor por defecto o uno hecho por nosotros). En el caso de un constructor creado debemos pasarle los parámetros. Objeto1=NombreClase(parametro1, parametro2,etc)

Desde fuera de la clase podemos acceder a los atributos escribiendo
NombreObjeto.NombreAtributo

MÉTODOS: Para crear un método basta con escribir def
NombreMetodo(self,parametro1,parametro2,etc):

Y luego dentro especificar que hace el método. Para llamar a este método debemos escribir
Objeto.NombreMetodo(parametro1,parametro2,etc)

```
Python-Codigos > DaltoEjercicio1.py > ...
1  class Estudiante:
2      def __init__(self,nombre,edad,grado) :
3          self.nombre=nombre
4          self.edad=edad
5          self.grado=grado
6      def Estudiar(self):
7          print(f"El estudiante {self.nombre} está estudiando")
8
9  nombre=(input("Ingrese su nombre "))
10 edad=(input("Ingrese su edad "))
11 grado=(input("Ingrese su grado "))
12
13 E1=Estudiante(nombre,edad,grado)
14 accion=input()
15 if(accion.lower()=="estudiar"):
16     E1.Estudiar()
```

Podemos utilizar **pass** para evitar que salte un error cuando hallamos creado un método que todavía no pensemos definir.

```
def saludar():
    pass
```

Los **métodos privados** son aquellos que están destinados a ser utilizados internamente dentro de una clase y no deben ser accedidos desde fuera de la clase. Aunque Python no tiene

verdaderos métodos privados como en otros lenguajes como C++, se sigue una convención de nomenclatura para identificarlos. Los métodos privados en Python se nombran con uno o dos guiones bajos _ como prefijo en el nombre del método. Esta convención indica que el método es privado y no debe ser accedido desde fuera de la clase. A pesar de que los métodos privados pueden ser accedidos desde fuera de la clase, la convención de nomenclatura indica que no deberían ser accedidos directamente.

HERENCIA: Cuando queremos hacer una clase que tenga todos los métodos y atributos de una, pero además que tenga otros extra, podemos realizar una herencia, en la que la clase hija (la nueva clase) hereda todo lo que tiene la clase padre.

Para **definir una herencia** debemos incluirla dentro de la definición de la clase hija de la siguiente forma.

```
class Empleado(Persona):
```

Al momento de **crear el constructor de la clase hija**, este debe recibir tanto los parámetros de la clase padre como los de la hija. Además, se debe llamar a una función llamada `super().__init__(parámetros_padre)` que se encarga de inicializar los atributos.

```
class Empleado(Persona):
    def __init__(self, nombre, edad, nacionalidad, trabajo, salario):
        super().__init__(nombre, edad, nacionalidad)
        self.trabajo = trabajo
        self.salario = salario
```

HERENCIA MÚLTIPLE: Cuando necesitamos que una clase herede atributos y métodos de más de una clase el proceso es muy similar.

Al crearla colocamos `class Hijo(Padre1, Padre2, etc):`

La mayor diferencia es que **en el constructor** cuando vamos a inicializar los atributos de las clases padres, no utilizamos la función `super`, sino que en vez de `super` escribimos el nombre de la clase padre. (Debemos hacerlo con cada clase padre)

```
class EmpleadoArtista(Persona, Artista):
    def __init__(self, nombre, edad, nacionalidad, habilidad, salario, empresa):
        Persona.__init__(self, nombre, edad, nacionalidad)
        Artista.__init__(self, habilidad)
        self.salario = salario
        self.empresa = empresa
```

SOBRECARGA DE MÉTODOS: Es posible sobrecargar métodos que se hayan heredado para que realicen una función diferente. Cuando la función se encuentra sobrecargada, podemos acceder a la sobrecarga llamándola como `self.funcion()` pero si queremos acceder al método del padre utilizamos `super().funcion()`.

```
def presentarse(self):
    return f'{self.mostrar_habilidad()}'
```

```
def presentarse(self):
    return f'{super().mostrar_habilidad()}'
```

MRO (METHOD RESOLUTION ORDER)

Hace referencia al orden con el que Python busca métodos y atributos en las clases. Esto tiene mucha incidencia en los casos en que las clases padres tengan un método o atributo llamado de igual forma.

Si sobrecargamos el método en la hija, al llamarlo estaríamos utilizando el de la clase actual (objeto al que hacemos referencia)

Si no es así, en jerarquía se encontrará primero el de la clase padre que primero hallamos declarado padre, luego el segundo, etc. Esto se complica cuando tenemos muchas clases heredadas, pero podríamos decir que se sigue la jerarquía como el recorrido en preorden de un árbol teniendo a la clase actual o hija como raíz. Esto solo funciona de otro modo si los padres de la clase comparte padre, en este caso la jerarquía se da por niveles.

Si no lo tienes claro puedes utilizar o hacer un print de la función `Clase.mro()` que te devuelve un texto con el orden jerárquico dentro de esa clase.

Si quieres utilizar la función de una clase que no queda primera según el orden puedes simplemente llamarla de la siguiente forma `ClaseSinJerarquía.funcion(OBJETO)`

COMO SABER SI UN OBJETO ES UNA INSTANCIA DE UNA CLASE/COMO SABER SI UNA CLASE HEREDA DE OTRA:

Para saber si una clase hereda de otra podemos utilizar la **función `issubclass(Hijo,Padre)`** que devuelve `true` si hijo hereda de padre y `false` si no lo hace. Luego para saber si un objeto es una instancia de cierta clase podemos utilizar **`isinstance(Objeto,Clase)`** que retorna `true` si objeto es una instancia de clase y `false` en caso contrario.

```
herencia = issubclass(EmpleadoArtista,Persona)
instancia = isinstance(roberto,Persona)
```

CIFRAR CONTRASEÑA: `hashlib.sha256(b"Nobody inspects the spammish repetition").hexdigest()`

Más info: <https://docs.python.org/3/library/hashlib.html>

`__str__` y `__repr__`: En Python, `__str__` y `__repr__` son métodos especiales utilizados para definir cómo se debe representar una instancia de una clase en forma de cadena. Aunque pueden parecer similares, tienen propósitos ligeramente diferentes:

`__str__`: Este método devuelve una representación legible para humanos de un objeto. Es utilizado por la función `str()` y por la función `print()` cuando se quiere obtener una versión "amigable" del objeto.

`__repr__`: Este método devuelve una representación sin ambigüedades del objeto, preferiblemente algo que permita crear una instancia igual al objeto original. Es utilizado por la función `repr()` y por el intérprete cuando se muestra un objeto como el resultado de una expresión.

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f'({self.x}, {self.y})'

    def __repr__(self):
        return f'Punto({self.x}, {self.y})'

p = Punto(3, 4)
print(str(p))    # Salida: (3, 4)
print(repr(p))   # Salida: Punto(3, 4)
```